

DC-SGD: Differentially Private SGD With Dynamic Clipping Through Gradient Norm Distribution Estimation

Chengkun Wei¹, Weixian Li, Chen Gong, and Wenzhi Chen², *Member, IEEE*

Abstract—Differentially Private Stochastic Gradient Descent (DP-SGD) is a widely adopted technique for privacy-preserving deep learning. A critical challenge in DP-SGD is selecting the optimal clipping threshold C , which involves balancing the trade-off between clipping bias and noise magnitude, incurring substantial privacy and computing overhead during hyperparameter tuning. In this paper, we propose Dynamic Clipping DP-SGD (DC-SGD), a framework that leverages differentially private histograms to estimate gradient norm distributions and dynamically adjust the clipping threshold C . Our framework includes two novel mechanisms: DC-SGD-P and DC-SGD-E. DC-SGD-P adjusts the clipping threshold based on a percentile of gradient norms, while DC-SGD-E minimizes the expected squared error of gradients to optimize C . These dynamic adjustments significantly reduce the burden of hyperparameter tuning C . The extensive experiments on various deep learning tasks, including image classification and natural language processing, show that our proposed dynamic algorithms achieve up to 9 times acceleration on hyperparameter tuning than DP-SGD. And DC-SGD-E can achieve an accuracy improvement of 10.62% on CIFAR10 than DP-SGD under the same privacy budget of hyperparameter tuning. We conduct rigorous theoretical privacy and convergence analyses, showing that our methods seamlessly integrate with the Adam optimizer. Our results highlight the robust performance and efficiency of DC-SGD, offering a practical solution for differentially private deep learning with reduced computational overhead and enhanced privacy guarantees.

Index Terms—Differential privacy, machine learning, deep learning, data privacy.

I. INTRODUCTION

DEEP learning (DL) has made significant strides in various fields, such as computer vision [1], [2], [3], [4], [5] and natural language processing [6], [7], [8], [9]. However, training

deep learning models using sensitive data, like images or DNA sequences, can pose serious privacy challenges, potentially exposing sensitive details [10], [11], [12], [13], [14], [15], [16], [17]. Differentially Private Stochastic Gradient Descent (DP-SGD) [18], which clips gradients and add noise to the clipped gradients to satisfy Differentially Privacy (DP), has become one of the most promising solutions for deep learning privacy.

The core principle of DP-SGD is to clip gradients and add noise to ensure DP. However, selecting an appropriate hyperparameters clipping threshold C remains a knotty issue [19], [20], [21], [22]. This difficulty arises from the fact that the optimal threshold varies across different scenarios [23], [24]. Specifically, the optimal value for C can change throughout the training process. Initially, a higher C might be beneficial to limit noise, but as training progresses and gradients typically become smaller, the same C could result in minimal clipping and an overestimation of noise impact. This trade-off between clipping and noise is critical: a higher C reduces clipping but increases noise, potentially degrading model accuracy, while a lower C reduces noise but increases bias, leading to information loss and reduced performance.

Moreover, **tuning the clipping threshold involves substantial computational overhead and can result in additional privacy leakage [25], [26]**. The hyperparameter tuning costs are significant. This process often requires multiple training runs with different C values to identify the optimal setting, each run being time-consuming and costly in terms of privacy. This highlights the importance of developing methods that can dynamically adjust C without extensive manual tuning.

A. Existing Solutions

Prior research [18], [23], [27], [28] has investigated the use of dynamic clipping thresholds to mitigate noise addition. For example, Abadi et al. [18] recommend setting the clipping threshold, C , as the median of gradient norms, and Du et al. [23] advocate for adjusting the threshold according to the training iteration. However, these methods either lack definitive privacy guarantees or require careful initial settings for the clipping threshold and its update rate [18], [23]. Another promising research direction involves dynamically setting the clipping threshold C based on the distribution of gradient norms. Andrew et al. [19] propose determining C using a hyperparameter p , representing a percentile of gradient norm distribution in federated learning with DP-FedAvg. While this approach eliminates the need for manual tuning of C , it

Received 11 August 2024; revised 16 February 2025; accepted 17 March 2025. Date of publication 18 April 2025; date of current version 5 May 2025. This work was supported in part by the National Natural Science Foundation of China under Grant No.62302441, in part by the Key Research and Development Plans of Guangxi Province under Grant No.AB22080077, and in part by the Information Technology Center of Zhejiang University and the Supercomputing Center of Hangzhou City University. The associate editor coordinating the review of this article and approving it for publication was Dr. Meng Li. (*Corresponding author: Wenzhi Chen.*)

Chengkun Wei and Wenzhi Chen are with the College of Computer Science and Technology, Zhejiang University, Hangzhou 310027, China (e-mail: weichengkun@zju.edu.cn; chenwz@zju.edu.cn).

Weixian Li is with the Ant Group, Hangzhou 310000, China (e-mail: liweixian.lwx@antgroup.com).

Chen Gong is with the School of Engineering, University of Virginia, Charlottesville, VA 22903 USA (e-mail: fzv6en@virginia.edu).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TIFS.2025.3557755>, provided by the authors.

Digital Object Identifier 10.1109/TIFS.2025.3557755

introduces other hyperparameters, p and η_C , thus still incurring more hyperparameter tuning costs.

In this paper, we aim to solve the problem: *How can we update the clipping threshold through private gradient norm distribution estimation, eliminating the need for hyperparameter tuning of C ?*

B. Our Method

We present **Dynamic Clipping DP-SGD (DC-SGD)**, a framework that dynamically updates the clipping threshold in differentially private SGD. Distinguishing from previous research, DC-SGD investigates the intricate relationship between the distribution of gradient norms and the clipping threshold, initially accessing the gradient norm distribution and then adaptively setting the clipping threshold. To ensure privacy, we use a histogram to estimate the distribution of gradient norms in a differentially private manner. Within this framework, DC-SGD is implemented through two mechanisms, DC-SGD-P and DC-SGD-E, which are elaborated as follows:

We first propose the mechanism DC-SGD with Percentile (DC-SGD-P), which is similar to Andrew et al. [19]. However, it has fewer parameters that need to be tuned. We adaptively set the clipping threshold during training to ensure that $p\%$ of gradients remain unclipped. In this way, we can enhance the stability of clipping by maintaining a consistent probability that each gradient is clipped throughout the training process. We shift the tuning from C to p , as our experimental findings suggest that the selection of P adheres to discernible patterns. What's more, we eliminate the cost of tuning η_C by adaptively setting C with the histogram.

The second mechanism is DC-SGD with Expected Squared Error (DC-SGD-E). We explore a noisy histogram to estimate this error under differential privacy, and dynamically construct a candidate set for the optimal clipping threshold. We dynamically set the clipping threshold by optimizing the expected squared error between the noisy and original gradients in SGD. By minimizing the expected squared error, we can reduce the impact of the privacy operation on individual gradients. Notably, *DC-SGD-E requires no additional hyperparameters to be tuned, thus eliminating the need for tuning C .*

We highlight the robust transferability of DC-SGD, as our methods are compatible with the Adam optimizer [29]. The Adam optimizer is particularly advantageous because it minimizes the need for tuning the learning rate (η). By integrating our methods with the Adam optimizer, we significantly reduce the complexity of simultaneously adjusting both the clipping threshold and the learning rate η . This integration enhances the overall efficiency and usability of our approach.

C. Evaluations

We perform a rigorous privacy analysis of DC-SGD-P and DC-SGD-E by converting it into equivalent DP-SGD, and provide rigorous convergence analysis. We also conduct comprehensive experiments to evaluate the effectiveness of DC-SGD across four datasets: MNIST [30], CIFAR10 [31], SVHN [32], and QNLI [33] on common models: small convolutional neural network, large backbone networks ResNet [34], and the pre-trained BERT-base model [35]. Experimental

results illustrate that our methods can achieve higher accuracy on most models and datasets compared to DP-SGD while saving the effort of tuning C .

For time cost, DC-SGD-E can achieve a $9\times$ speedup on hyperparameter tuning. For model performance, we consider the privacy cost of tuning and use RDP [36] to account for the privacy cost. Compared to other baselines [18], [19], [23], DC-SGD-E achieve an improvement of 10.62% on CIFAR10 using ResNet34 with $\epsilon = 2$, while DC-SGD-P achieve a 2.13% improvement for SVHN on ResNet34 with $\epsilon = 2$. This improvement becomes more pronounced compared to other methods requiring additional hyperparameter tuning. And if we ignore the privacy cost of tuning like previous work [19], [23], [37], [38], and compare the performance of different methods [18], [19], [23] with selectively chosen hyperparameter combination, DC-SGD-P and DC-SGD-E can still achieve a comparable or better performance compared to other baselines.

To sum up, we make the following contributions:

- We propose DC-SGD that determines a dynamic clipping threshold C by examining the gradient norm distribution. We instantiate it to two mechanisms and tackle the intricate issue of adjusting the clipping threshold.
- We conduct rigorous theoretical privacy analysis of DC-SGD by converting it to equivalent DP-SGD and perform a comprehensive theoretical analysis to ascertain the convergence properties of DC-SGD. The theoretical analysis illustrates the privacy and utility of our methods.
- Comprehensive experiments across image classification and natural language processing tasks present that both DC-SGD-P and DC-SGD-E outperform or match the performance of the vanilla DP-SGD while reducing the time and privacy cost of extensive hyperparameter tuning.

II. BACKGROUND AND CHALLENGES

A. Differential Privacy

Definition 1 ((ϵ, δ) -Differential Privacy [39]): A mechanism or algorithm $M : \mathcal{D} \rightarrow \mathbb{R}^d$ satisfies (ϵ, δ) -differential privacy if, for any pair of neighboring datasets $D, D' \in \mathcal{D}$, and any subset $S \subseteq \mathbb{R}^d$, it holds that:

$$\Pr[M(D) \in S] \leq e^\epsilon \Pr[M(D') \in S] + \delta$$

where ϵ is the privacy budget, measuring the similarity of outputs on neighboring datasets, with a smaller ϵ meaning better privacy protection. δ denotes the probability that the privacy guarantee fails.

The concept of neighboring datasets varies by context, affecting the degree of DP. In example-level DP, two datasets are considered neighboring if they differ by a single example. In user-level DP, a single user may contribute multiple examples, resulting in neighboring datasets differing by several examples [19], [40]. *This paper focuses on example-level DP to ensure the privacy of individual examples, and the DP sensitivity is defined as follows:*

Definition 2 (Sensitivity of a function f [39]): Given two neighboring datasets $D, D' \in \mathcal{D}$, and a function $f : \mathcal{D} \rightarrow \mathbb{R}^d$.

The $\mathcal{L}_m$ -sensitivity of function f is defined as the maximum value by which f changes with a single individual,

$$\Delta_{nf} = \max_{D, D'} \|f(D) - f(D')\|_m$$

where $\|\cdot\|_m$ denotes the $\mathcal{L}_m$ norm. The sensitivity measures the maximum change in the query output caused by altering a single example.

We leverage Rényi DP (RDP) [36] to get a tighter privacy analysis under composition [19]. RDP is fundamentally grounded on the concept of Rényi divergence. Given two probability distributions P and Q defined over $\mathbb{R}$, the Rényi divergence of order $\alpha > 1$ is given by

$$D_\alpha(P\|Q) = \frac{1}{\alpha - 1} \log \mathbb{E}_{x \sim Q} \left(\frac{P(x)}{Q(x)} \right)^\alpha,$$

where $P(x)$ denotes the density of P at point x . In this expression, the logarithm is the natural logarithm, and the notation $x \sim Q$ signifies that x is drawn from the distribution Q . Then, RDP is defined as follows,

Definition 3 ((α, ρ) -RDP [36]): A mechanism $f : \mathcal{D} \rightarrow \mathbb{R}^d$ is said to possess ρ -Rényi differential privacy of order α , denoted as (α, ρ) -RDP, if for any neighboring datasets $D, D' \in \mathcal{D}$, the following condition is met: $D_\alpha(f(D)\|f(D')) \leq \rho$.

Mironov et al. [36] demonstrate that RDP provides a more stringent privacy analysis under composition than (ϵ, δ) -DP. Furthermore, RDP can be translated into (α, ϵ) -DP using the subsequent lemma.

Lemma 1 (From (α, ρ) -RDP to (ϵ, δ) -DP [41]): If a mechanism $\mathcal{M}$ adheres to (α, ρ) -RDP, then it also satisfies (ϵ, δ) -DP for any $0 < \delta < 1$ where $\epsilon = \rho + \log \left(\frac{\alpha-1}{\alpha} \right) - \frac{\log \delta + \log \alpha}{\alpha-1}$.

Therefore, we can use RDP for privacy analysis and subsequently convert it into (ϵ, δ) -DP to achieve a more refined privacy assessment. DP-SGD can be viewed as a composition of the Subsampled Gaussian Mechanism (SGM). Mironov et al. [42] give the analysis of the SGM under RDP.

Lemma 2 (Privacy Analysis of SGM [42]): Let $SG_{q, \sigma}$ be the SGM with sample rate q and Gaussian noise $\mathcal{N}(0, \sigma^2 S_f^2 \mathbb{I})$ for some function f with sensitivity S_f . Then, $SG_{q, \sigma}$ satisfies (α, ρ) -RDP with the condition of

$$\rho \leq D_\alpha((1-q)\mathcal{N}(0, \sigma^2) + q\mathcal{N}(1, \sigma^2)\|\mathcal{N}(0, \sigma^2)).$$

In this study, we use Lemma 2 to calculate the overall privacy budget. It is worth noting that the Lemma 2 are built based on Poisson sampling. Throughout this paper, all sampling processes adhere to Poisson sampling, where each example has an equal probability of being sampled.

B. DP Stochastic Gradient Descent

DP-SGD [18] is the predominant method for training neural networks with differential privacy guarantees using SGD. The core principle of DP-SGD involves introducing noise to the gradients during each iteration. To effectively add this noise, DP-SGD first bounds the sensitivity of the gradients. Due to the difficulty in predicting gradient norm distributions in deep learning, DP-SGD sets an upper bound C on the $\mathcal{L}_2$ norm of the gradients, expressed as:

$$\text{Clip}(g, C) = g / \max \left(1, \frac{\|g\|_2}{C} \right) \quad (1)$$

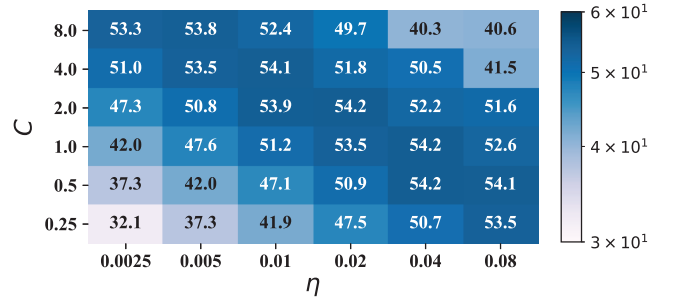


Fig. 1. Classification accuracy of ResNet18 trained on the CIFAR10 dataset, using varying clipping thresholds C , learning rates η , $\epsilon = 8$ and SGD optimizer within the DP-SGD framework.

This method limits the influence of any single instance on C , effectively bounding the $\mathcal{L}_2$ -sensitivity by C . DP-SGD introduces Gaussian noise with a standard deviation proportional to C to the aggregated gradients. The algorithm then averages these gradients and updates the parameters as follows:

$$\theta_{t+1} = \theta_t - \frac{\eta}{B} \left(\sum_{i \in \mathcal{B}_t} \text{Clip}(g_{t,i}, C) + \mathcal{N}(0, \sigma^2 C^2 \mathbb{I}) \right) \quad (2)$$

' B ' denotes the training batch size. The DP-SGD approach can be viewed as a composition of multiple SGMs. In practice, it is common to use Lemma 2 for a more stringent privacy analysis under RDP, and subsequently convert this into (ϵ, δ) -DP [19], [43], [44]. We adopt this strategy in our paper.

C. Extra Privacy Cost of Hyperparameter Tuning C

When implementing DP-SGD to train deep learning models with differential privacy, tuning hyperparameters such as the learning rate, batch size, and clipping threshold is essential. Typically, tuning the hyperparameters entails training models multiple times with various combinations of hyperparameters, which incur extra privacy costs. *Adjusting C incurs computational overhead and increases privacy leakage risks.* As shown in Figure 1, a two-dimensional grid search for the optimal learning rate η and C (extendable to other dimensions like noise scheduling or model architecture) demands extensive model training across various hyperparameter settings, consuming significant time and computational resources. Additionally, multiple training runs for hyperparameter tuning lead to further privacy leakage [25], [26].

Liu et al. [25] proposed a DP hyperparameters tuning algorithm (we call it LT) that uses random stopping with a hard limit T on the number of iterations. We present the privacy analysis of LT as follows:

Theorem 1 (Privacy Analysis of LT [25]): We consider any $\gamma \in [0, 1]$ and $\delta_2 > 0$, and define T as $\frac{1}{\gamma} \ln \frac{1}{\delta_2}$. Assuming that each iteration of DP-SGD is (ϵ_1, δ_1) -differentially private, the LT algorithm then achieves (ϵ', δ') -differential privacy. ϵ' is defined as $3\epsilon_1 + 3\sqrt{2\delta_1}$, and δ' is calculated as $3\sqrt{2\delta_1}T + \delta_2$.

Theorem 1 details the privacy cost associated with the LT algorithm. As the number of hyperparameter combinations to be tuned increases, the value of T also rises. Under a constant privacy budget, this necessitates a smaller δ_1 . Consequently, a

reduced δ_1 requires a smaller ϵ_1 , which in turn can affect the performance of the model.

D. Challenges of Dynamic Clipping

A static C may initially seem adequate but can become suboptimal as training progresses. This is because gradients typically diminish over time in standard SGD, causing a previously ideal high C to result in minimal clipping and excessive noise [23], [45]. However, determining an optimal clipping threshold C in DP-SGD presents several significant challenges:

1) *Trade-Offs Between Clipping and Noise*: The selection of C necessitates balancing the bias introduced by clipping against the magnitude of noise, which scales with C . In scenarios with constrained privacy budgets, a higher C reduces bias but may degrade performance due to increased noise. A lower C may induce excessive bias from over-clipping, leading to significant information loss and diminished model effectiveness.

2) *Lack of Private Prior Knowledge*: The distribution of gradient norms is often private and unknown a priori, complicating the prediction of the appropriate clipping threshold. This lack of prior knowledge necessitates heuristic or empirical methods to determine C , which may not always yield optimal results.

3) *Hyperparameter Tuning Costs*: Tuning the clipping threshold incurs significant computational overhead and additional privacy leakage (Section II-C). Identifying the optimal C often requires multiple training runs, each time-consuming and privacy-costly. This highlights the need for methods that can dynamically adjust C without extensive manual tuning.

III. DC-SGD DESIGN

A. Overview

To optimize the clipping threshold C while preserving privacy, we introduce a novel method that employs differentially private histograms to analyze gradient norm distributions. The key advantage of using histograms is their sensitivity of 1. Meanwhile, the histogram can provide a comprehensive view of the whole distribution since a histogram is a direct description of the distribution.

We introduce a framework named **Dynamic Clipping DP-SGD** (DC-SGD), which dynamically adjusts the clipping threshold as outlined in Algorithm 1. Unlike traditional DP-SGD, DC-SGD incorporates two key processes: *histogram construction* (Line 12) and *adaptive adjusting C* (Line 14). DC-SGD strategically allocates the privacy budget between model training and updating C , as detailed in Section IV-A. During histogram construction, data points are categorized into bins, with outliers allocated to the last bin, and differential privacy is maintained through noise injection. We propose two specific strategies for adjusting the clipping threshold C : DC-SGD with Percentile (DC-SGD-P) and DC-SGD with Expected Squared Error (DC-SGD-E).

1) *Dcsgd With Percentile (DC-SGD-P)*: We introduce a single hyperparameter, p , to dynamically adjust the clipping threshold C during training (Detailed in Section III-C). This

Algorithm 1 DC-SGD

Input: Examples $\{x_1, \dots, x_N\}$, loss function $\mathcal{L}(\theta) = \frac{1}{N} \sum_i \mathcal{L}(\theta, x_i)$. Parameters: learning rate η , total noise multiplier σ , noise multiplier for histogram σ_H , expected batch size B , initial clipping threshold C_0 , initial histogram range R_0 , count of histogram bins b .

1 **Initialize:** θ_0 randomly $\sigma_T \leftarrow (\sigma^{-2} - \sigma_H^{-2})^{-1/2}$ **for**
 $t = 0$ **to** T **do**
2 Take a random batch $\mathcal{B}_t$ with sampling probability B/N
3 **Compute gradient**
4 For each $i \in \mathcal{B}_t$, compute $g_{t,i} \leftarrow \nabla_{\theta_t} \mathcal{L}(\theta_t, x_i)$
5 **Compute Gradient Norms**
6 For each $i \in \mathcal{B}_t$, compute $g_{t,i} \leftarrow \|g_{t,i}\|$ and put it into G_t
7 **Private Descent**
8 Update parameters θ using Equation (2)
9 **Construct Histogram**
10 $\tilde{H}_t \leftarrow \text{BuildHistogram}(G_t, R_t, b)$
11 **Adjust Clipping Threshold**
12 $C_{t+1}, R_{t+1} \leftarrow \text{Adaptive}(\tilde{H}_t, C_t, R_t, b)$
13 **end**
Output: model parameters θ_{T+1}

approach ensures that $p\%$ of the gradients remain unclipped, effectively clipping with a probability of $1 - p$. Distinct from previous methods [18], [19], [22], DC-SGD-P requires tuning only this additional hyperparameter and eliminates the need for an external public dataset. Moreover, it guarantees that the adjustment of C adheres to DP.

2) *Dcsgd With Expected Squared Error (DC-SGD-E)*: We further developed a method that autonomously determines a clipping threshold C without additional hyperparameter tuning (Detailed in Section III-D). By adaptively adjusting C to minimize the expected error of individual gradients (as defined in Equation 3), DC-SGD-E alleviates the effects of clipping and noise on individual gradients within the bounds of DP. This pioneering approach eliminates the need for hyperparameter adjustments, significantly enhancing both time efficiency and privacy protection.

The newly determined clipping threshold is applied in subsequent iterations for two primary reasons: First, as stated in Theorem 2, our methodology involves the simultaneous publication of both the gradient and a one-hot histogram vector. Adjusting the current gradient with a newly adapted threshold post-histogram analysis would contravene this assumption. Second, in practical implementations, a small physical batch size is frequently used to simulate a larger logical batch size using gradient accumulation techniques. Applying the new clipping threshold within the same iteration would necessitate retaining all unclipped gradients until the logical batch size is achieved, thus restricting the utility of gradient accumulation methods.

Algorithm 2 HistogramBuild

Input: Unclipped gradients' norms
 $\mathcal{G} = \{G_1, G_2, \dots, G_{|\mathcal{B}_t|}\}$, histogram range R_t ,
bin count b , noise multiplier σ_H

1 **Initialize** $H_t[b] = \{0, \dots, 0\}$

2 **for** $G_i \in \mathcal{G}$ **do**

3 $index \leftarrow \max(n-1, \lfloor \frac{bG_i}{R_t} \rfloor)$

4 $H_t[index] \leftarrow H_t[index] + 1$

5 **end**

6 $\tilde{H}_t \leftarrow H_t + \mathcal{N}(0, \sigma_H^2 \mathbb{I})$

Output: Noisy histogram $\tilde{H}_t$

Algorithm 3 Adaptive C With Percentile

Input: noisy histogram $\tilde{H}_t$, bin count b , current
clipping threshold C_t , histogram range R_t ,
percentile p .

1 **Initialize** $S = 0$

2 $S' \leftarrow \sum_{i=0}^{b-1} \tilde{H}_t[i]$

3 **for** $i \in [b]$ **do**

4 $S += \tilde{H}_t[i]$

5 Use the middle of the current bin as C_{t+1}

6 **If** $S \geq pS'$, **break**

7 **end**

8 $R_{t+1} = 2C_{t+1}$

Output: C_{t+1}, R_{t+1}

B. Histogram Construction

Accessing precise gradient distribution information is crucial for adaptively determining the clipping threshold C . Algorithm 2 outlines the construction of a gradient norm histogram in each training iteration under DP constraints. Data points that exceed the predefined range are allocated to the last bin. Notably, we address the inherent bias associated with the range R in DC-SGD-P and DC-SGD-E through adaptive adjustment strategies.

For DP compliance, noise is added to each histogram bin. The sensitivity of histogram publication is 1, ensuring minimal changes caused by adding or removing a single example. At each iteration, gradients and corresponding histogram information are published with Gaussian noise for privacy. Detailed privacy analysis is provided in Section IV-A. In practice, a small portion of the privacy budget is allocated to histogram publication, and σ_H is set to minimize changes in gradient noise without incurring extra tuning costs. Additionally, a sufficiently large batch size is recommended to mitigate bias and reduce the impact of noise.

C. DC-SGD With Percentile

In DC-SGD, the clipping threshold determined in the current iteration is applied to clip gradients in the subsequent iteration. Ideally, setting a percentile-based threshold should involve considering the entire training dataset, but this incurs significant time and privacy costs. In DP-SGD, where each training example is equally likely to be sampled for constructing a mini-batch. Therefore, DC-SGD-P estimates the

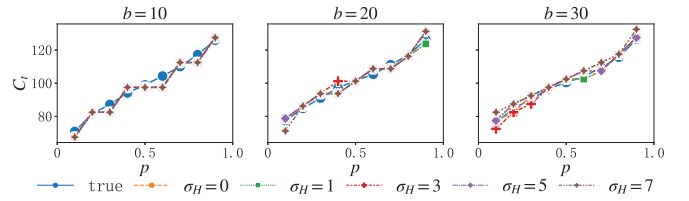


Fig. 2. C_t according to different percentile p on a set of 256 random synthetic data drawn from $\mathcal{N}(100, 20^2)$ with different histogram construction and noise level. Each subfigure uses a different histogram bin count b and generates data separately.

Percentile using the sampled batch and leverages the histogram of gradient norms instead of accessing the complete dataset, as presented in Algorithm 3. DC-SGD-P consists of two steps: (1) determining the clipping threshold based on the gradient norm distribution to ensure that $p\%$ of gradients remain unclipped; (2) adjusting the histogram range.

1) *Estimate the Percentile:* To estimate the Percentile, we calculate the midpoint of each bin as the estimated norm value. We then accumulate counts starting from the leftmost bin and continue to the right until the cumulative total exceeds $p\%$ of the overall norm count, which is determined by summing across all bins. Disregarding the added noise in the histogram, the true and estimated percentiles are within the same bin, yielding a maximum error of $\lfloor \frac{R}{2b} \rfloor$. Given the iterative nature of the C adjustment, this error remains acceptable with careful selection of R and b . Note that b and R do not incur additional tuning costs. Figure 2 depicts our method's estimation results on synthetic data. With a suitable range ($R = 150$ covering most data), our approach closely approximates the true result, demonstrating low sensitivity to σ_H and b . More results on true datasets can be found in Section VI-C.

2) *Adjust the Range of Histogram:* Determining the histogram's range is crucial for precisely identifying desired gradient norms. If the range is too narrow, outliers cluster in the last bin, resulting in inaccurate estimations of C . Conversely, an overly broad range may produce less precise results. Additionally, the variability in gradient norms across training epochs, models, and datasets complicates setting a fixed range that suits all scenarios. To this end, we propose an adaptive method that dynamically determines the histogram range based on the current clipping threshold C . Specifically, we configure the histogram within a $[0, 2C]$ range, where C denotes the existing clipping threshold. If the histogram fails to yield a new clipping threshold corresponding to the desired Percentile, indicated when the output is the median of either the first or last bin, this output is then adopted as the new current clipping threshold, prompting an adjustment of the range for improved histogram accuracy in subsequent iterations. Given the exponential increase or decrease in the histogram range, only a minimal number of repetitions are required to rapidly determine an appropriate range. It is important to note that while this method starts with an initial value of C , it can be arbitrarily set as the algorithm is designed to refine it effectively.

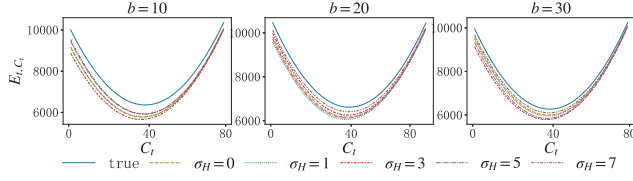


Fig. 3. E_{t,C_t} of different C_t on a set of 256 random synthetic data from $\mathcal{N}(100, 20^2)$ with different histogram structures and noise levels. To compute the variance term, $\sigma_T = 1, B = 256, d = 100000$. The histograms all have $R = 120$. Each subfigure uses a different histogram bin count b and generates data separately.

D. DC-SGD With Expected Squared Error

While the method in Section III-C eliminates the need to tune the clipping threshold C , it introduces a new dataset-dependent hyperparameter, Percentile p . In this subsection, we devise a method that autonomously determines a clipping threshold without additional hyperparameter tuning.

1) *Algorithm Design*: DP-SGD updates the model parameters by $\frac{1}{B} (\sum_{i \in \mathcal{B}_t} \text{Clip}(g_{t,i}, C_t) + \mathcal{N}(0, \sigma_T^2 C_t^2 \mathbb{I}^d))$, where $\text{Clip}(g_{t,i}, C_t)$ means clipping the gradient with clipping threshold C_t at iteration t . Focusing on a single gradient $g_{t,i}$, the operation on it is equivalent to $\tilde{g}_{t,i} = \text{Clip}(g_{t,i}, C) + \mathcal{N}(0, \frac{\sigma_T^2 C^2 \mathbb{I}}{|\mathcal{B}_t|^2})$, where $\tilde{g}_{t,i}$ is the private estimation of gradient $g_{t,i}$. We define $\bar{g}_{t,i} = \text{Clip}(g_{t,i}, C)$, and consider the randomness in noise and data sampling at iteration t , the expected squared error between $g_{t,i}$ and $\tilde{g}_{t,i}$ can be expressed as:

$$\begin{aligned}
 E_{t,C} &= \mathbb{E}[(\tilde{g}_{t,i} - g_{t,i})^2] \\
 &= \mathbb{E}[(\tilde{g}_{t,i} - \bar{g}_{t,i})^2] + 2\mathbb{E}[(\tilde{g}_{t,i} - \bar{g}_{t,i})(\bar{g}_{t,i} - g_{t,i})] \\
 &\quad + \mathbb{E}[(\bar{g}_{t,i} - g_{t,i})^2] \\
 &\stackrel{(i)}{=} \mathbb{E}[(\tilde{g}_{t,i} - \bar{g}_{t,i})^2] + \mathbb{E}[(\bar{g}_{t,i} - g_{t,i})^2] \\
 &\stackrel{(ii)}{=} \frac{\sigma_T^2 C^2 d}{|\mathcal{B}_t|^2} + \left(\frac{1}{N} \sum_{j=1}^N \|g_{t,j} * \min\left(1, \frac{C}{\|g_{t,j}\|}\right) - g_{t,j}\|^2 \right) \\
 &= \underbrace{\frac{\sigma_T^2 C^2 d}{|\mathcal{B}_t|^2}}_{\text{Variance}} + \underbrace{\left(\frac{1}{N} \sum_{j=1}^N \max(\|g_{t,j}\| - C, 0)^2 \right)}_{\text{Bias}} \quad (3)
 \end{aligned}$$

(i) follows from $\tilde{g} - \bar{g} = \mathcal{N}(0, \frac{\sigma_T^2 C^2 \mathbb{I}}{|\mathcal{B}_t|^2})$, indicating that this term is independent of the gradient and each of its dimensions has an expected value of 0. As for (ii), we note that each noise dimension is independent and the variance is $\frac{\sigma_T^2 C^2}{|\mathcal{B}_t|^2}$. Given that the noise has d dimensions, the total variance caused by noise is $\frac{\sigma_T^2 C^2 d}{|\mathcal{B}_t|^2}$. For the bias caused by clipping, we calculate the expected value using the average overall gradients, as each example is uniformly sampled from the entire dataset.

In Equation (3), the larger C results in stronger noise, which increases variance. However, a larger C simultaneously reduces bias, as fewer gradients are clipped, and vice versa. The bias and variance go in opposite directions, and an optimal C should exist. Figure 4 illustrates the expected squared error for different clipping thresholds across various datasets on

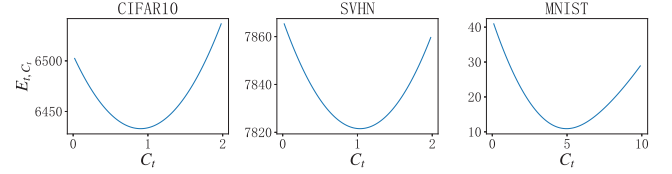


Fig. 4. E_{t,C_t} of different C_t at some iteration for CIFAR10 on ResNet18, SVHN on ResNet34, MNIST on CNN. Privacy budget ($\epsilon = 8, \delta = 1/|D|$), batch size $B = 256$, clipping threshold $C = 1$, using default Adam optimizer.

ResNet18, supporting our analysis. We can fine-tune C by calculating the expected squared error and selecting the C value that minimizes this error.

To calculate the expected squared error for various clipping thresholds C while maintaining a low differential privacy cost, we have modified Equation (3) accordingly. First, in DP-SGD, we apply Poisson sampling to construct the mini-batch; thereby, the mini-batch's data count differs among iterations. The actual count will leak the privacy. Moreover, the new clipping threshold at the current iteration will be utilized at the next iteration, and the count of the next iteration is unpredictable. Therefore, we use expected batch size B instead of $|\mathcal{B}_t|$ in variance term. Second, to save privacy and time, we can estimate the expectation of bias terms on the mini-batch because each example in the mini-batch is sampled uniformly from the whole dataset. In summary, we get:

$$\begin{aligned}
 \mathbb{E}[(\tilde{g}_{t,i} - g_{t,i})^2] &= \underbrace{\frac{\sigma_T^2 C_t^2 d}{B^2}}_{\text{Variance}} \\
 &\quad + \underbrace{\left(\frac{1}{|\mathcal{B}_t|} \sum_{j \in \mathcal{B}_t} \max(\|g_{t,j}\| - C_t, 0)^2 \right)}_{\text{Bias}} \quad (4)
 \end{aligned}$$

To determine an optimal value for C with minimal expected squared error, as described in Equation (3), it is necessary to obtain the norms of the gradients to calculate the bias term. To ensure DP, we use a histogram-based estimation (similar to what we did for DC-SGD-P) of the norm distribution and subsequently perform an approximate computation of the bias term using the estimated results. Specifically, as shown in Algorithm 4, it consists of five steps: (1) get the estimation of data count at the current iteration; (2) create a candidate set of potential values for the clipping threshold C ; (3) approximate computation of the expected squared error using the noisy histogram and selection of the optimal C with the smallest expected squared error; (4) verification if the candidate set contains a value of C in proximity to the optimal one. If not, the algorithm is repeated; (5) adjust the histogram range.

2) *Estimation of E_{t,C_t}* : To estimate E_{t,C_t} , we use the mid-point of each bin in the noisy histogram as the estimated norm for that bin. Additionally, our noisy histogram designates the last bin for outliers. While these two practices might introduce a certain degree of bias in estimating the bias term, our primary focus is on the relative magnitudes of E_{t,C_t} across different C_t values, rather than on exact values. Figure 3 illustrates the variation of E_{t,C_t} for a set of random synthetic data under

Algorithm 4 Adaptive C With Minimizing Expected Squared Error

Input: noisy histogram $\tilde{H}_t$, bin count b , current clipping threshold C_t , histogram range R_t .

```

1  $S' \leftarrow \sum_{i=0}^{b-1} \tilde{H}_t[i]$ 
  // Computing Error and Selecting  $C$ 
2 while True do
3    $S \leftarrow \emptyset$ 
4   For  $i \in [1, 20]$ ,  $S \leftarrow S \cup (iC_t/10)$ 
5   for  $C' \in S$  do
6     Compute the bias term  $B$  by using the median
       of each bin to estimate the norms in it.
7     Compute  $E_{C'}$  with Equation (4)
8   end
9    $C_{t+1} \leftarrow \underset{C'}{\operatorname{argmin}}(E_{C'})$ 
10  Repeat by setting  $C_t = C_{t+1}$  if  $C_{t+1}$  is the
     boundary of candidates
11 end
  // Adjusting the range of histogram
12 if  $\tilde{H}_t[b-1] \geq 0.5S'$  then
13    $R_{t+1} = 2R_t$ 
14 end
15 else if  $\sum_{i=b/2}^{b-1} \tilde{H}_t[i] \leq S'/b$  then
16    $R_{t+1} = 0.5R_t$ 
17 end
Output:  $C_{t+1}, R_{t+1}$ 

```

different histogram structures and noise levels as C changes. $R = 120$ implies that a significant amount of data lies outside the histogram range, causing the overall estimate to be relatively smaller compared to the true value. However, regardless of the settings for σ_H and b , our approximate calculation curve closely tracks the true curve. This consistency ensures that we can determine a nearly optimal clipping threshold. The above analysis indicates that our method exhibits low sensitivity to the σ_H , b , and R , thus obviating additional tuning costs.

3) *Construction of Candidate Set:* Compared to DC-SGD-P, the main distinction lies in constructing the candidate set for C . Considering the infinite possible values for C , we employ a strategy to build and select from a finite set of candidates. Given the characteristic ‘U’ shape of the expected squared error curve to C , as depicted in Figure 4, we implement the exponential growth strategy outlined in Algorithm 3. Specifically, we generate 20 candidate clipping thresholds based on the current value of C , computed as $0.1C, 0.2C, \dots, 2C$. We then compute the expected error for each candidate. Notably, this computation, based on the noisy histogram, preserves privacy by adhering to the post-processing property of differential privacy. If the candidate yielding the smallest expected squared error is either $0.1C$ or $2C$, it is established as the new clipping threshold. This action triggers repetitive execution of the selection process, ensuring the algorithm explores a broad range of thresholds and ultimately converges to a value close to the optimal C , situated at the lowest point of the curve.

4) *Adjust the Range of Histogram:* We opt for a histogram range of $[0, 2C]$ as we focus primarily on the norms within the

specific Percentile of gradients’ norms. While our method is not overly sensitive to the exact choice of R , since our concern lies more with the relative magnitudes of E_{t,C_t} for different C_t values than with precise values, we still aim to adjust the range adaptively to accommodate the variability of gradient norms across various scenarios. For setting the range, one could either conduct experiments on a similar public dataset or employ an adaptive approach as specified in Algorithm 4, which leverages the similarity in gradient distributions across neighboring training iterations. Given that DC-SGD-E is not highly sensitive to the histogram range r , as shown in Figure 3, we employ a “cautious updates” strategy to refine the range: (1) if the count in the rightmost bin exceeds $0.5|\mathcal{B}_t|$, suggesting the range is too narrow for accurate estimation, we adjust R_{t+1} to be double the current R_t ; (2) to avoid an extensive range, if the sum of counts in the right half of the bins is less than $\frac{|\mathcal{B}_t|}{n}$, we reduce R_{t+1} to half of R_t .

IV. THEORETICAL ANALYSIS

A. Privacy Analysis

In DC-SGD, we split the total noise multiplier σ into two parts: σ_T for training and σ_H for clipping threshold updating. Then, the privacy cost of DC-SGD with σ_T and σ_H is equivalent to vanilla DP-SGD with σ . The related theorem and proof are presented as follows:

Theorem 2: Algorithm 1 uses Gaussian noise with a noise multiplier of σ_T to perturb the gradient, and noise with a noise multiplier of σ_H to perturb the histogram. This approach is equivalent to the standard DP-SGD, using a noise multiplier σ for gradient perturbation. This equivalence holds under the calculation of privacy costs when:

$$\sigma_T = (\sigma^2 - \sigma_H^2)^{-1/2} \quad (5)$$

Proof: In DC-SGD, each example, in addition to publishing its own gradient, also releases the gradient norm falling within which bin of the histogram that can be represented using one-hot encoding $H_{t,i}$. And the histogram can be constructed by $H_t = \sum_{i \in \mathcal{B}_t} H_{t,i}$. To analyze the noise added separately to the gradient and the histogram as a whole, we make a conceptual modification to the algorithm that does not influence the privacy properties like [19]. Instead of publishing $(g_{t,i}, H_{t,i})$, example i publish $(\tilde{g}'_{t,i}, H'_{t,i}) = (\tilde{g}_{t,i}/\sigma_T C, H_{t,i}/\sigma_H)$, where $\tilde{g}_{t,i}$ is the gradient after clipping. Let d represent the dimension of gradient, and the computation of averaged noise gradient and aggregated noise histogram can be written as:

$$\tilde{g} = \frac{\sigma_T C}{B} \left(\sum_{i \in \mathcal{B}_t} \tilde{g}'_{t,i} + \mathcal{N}(0, \mathbb{I}^d) \right), \tilde{H} = \sigma_H \left(\sum_{i \in \mathcal{B}_t} H'_{t,i} + \mathcal{N}(0, \mathbb{I}^b) \right) \quad (6)$$

Then, the noise addition can be rearranged as follows:

$$\sum_{i \in \mathcal{B}_t} (\tilde{g}'_{t,i}, H'_{t,i}) + \mathcal{N}(0, \sigma^2 \Delta^2 \mathbb{I}^{d+b}), \sigma \Delta = 1 \quad (7)$$

The above Equation is equal to DP-SGD with gradients in $d+b$ dimensions. σ is the noise multiplier and Δ is the clipping

threshold that is the upperbound of $\|\tilde{g}'_{t,i}, H'_{t,i}\|$. Then we handle Δ :

$$\begin{aligned} \|(\tilde{g}'_{t,i}, H'_{t,i})\| &= \sqrt{\mathbf{g}'_{t,i,1}{}^2 + \dots + \mathbf{g}'_{t,i,d}{}^2 + H_1'^2 + \dots + H_b'^2} \\ &\stackrel{(i)}{=} \sqrt{\|\tilde{\mathbf{g}}'_{t,i}\|_2^2 + H_k'^2} \\ &\stackrel{(ii)}{\leq} \sqrt{\left(\frac{C}{\sigma_T C}\right)^2 + \left(\frac{1}{\sigma_H}\right)^2} \\ &= \sqrt{\left(\frac{1}{\sigma_T}\right)^2 + \left(\frac{1}{\sigma_H}\right)^2} = \Delta \end{aligned} \quad (8)$$

In the above derivation, (i) is due to the definition of $\mathcal{L}_2$ norm and $H_{t,i}$ is a one-hot vector so that only H'_k is not 0; (ii) is due to $\tilde{g}_{t,i}$ is clipped by C . Then we can get $\sigma_T = (\sigma^{-2} - \sigma_H^{-2})^{-1/2}$ utilizing $\sigma\Delta = 1$.

For the privacy analysis of DC-SGD-P and DC-SGD-E. We have shown the privacy analysis of DC-SGD can be converted to that of DP-SGD. Therefore, DC-SGD-P is differentially private. Then, in both DC-SGD-P and DC-SGD-E, the extra computation is based on the published noisy histogram. With the post-processing property of DP, the additional computation does not incur any privacy cost. In summary, both DC-SGD-P and DC-SGD-E satisfy DP, and the privacy cost can be computed by converting it into an equivalent DP-SGD with Theorem 2.

B. Convergence Analysis

In this section, we present the convergence analysis of our proposed methods for vanilla SGD optimizer. Inspired by the work of Du et al. [23], we adopt a similar convergence analysis framework. The convergence analysis results for DC-SGD-P are summarized in Theorem 3. The convergence analysis aims to demonstrate the impact of our dynamic clipping operation on convergence instead of directly illustrating the superiority of our methods on convergence.

Theorem 3 (Convergence Analysis of DC-SGD-P): Let $\mathcal{L}(\theta)$ denote the loss function, and $g_t = \frac{1}{N} \sum_{i=0}^N g_{t,i} = \nabla \mathcal{L}(\theta_t)$ represent the actual gradient of $\mathcal{L}(\theta_t)$, where N is the dataset size. The estimated gradient after clipping at iteration t is represented as $\tilde{g}_t = \frac{1}{B} \sum_{i \in \mathcal{B}_t} \text{Clip}(g_t, i, C)$, with B being the expected batch size. Constants L and G are given (from the assumption that L -smooth and gradient norm is bounded), d is the dimension of gradients, p is the Percentile with $p' = 1 - p$, T is the total number of iterations, and $q = B/N$ is the sampling rate. If we disregard the bias from histogram estimation and the privacy cost of the histogram and set $\sigma_H = \sigma$. Then, for any $\epsilon < c_1 \gamma^2 T$, $\delta > 0$, $\sigma = c_2 \frac{q \sqrt{T \ln(1/\delta)}}{\epsilon}$, DC-SGD-P satisfies the (ϵ, δ) -DP guarantee, while DC-SGD-E satisfies (ϵ, δ) -DP guarantee, we have:

$$\begin{aligned} &\frac{1}{T} \mathbb{E} \left[\sum_{t=1}^T \|\nabla \mathcal{L}(\theta_t)\|^2 \right] \\ &\leq \frac{1}{T\eta} (\mathbb{E}[\mathcal{L}(\theta_1)] - \mathbb{E}[\mathcal{L}(\theta_{T+1})]) + \frac{L\eta}{T} \sum_{t=1}^T \mathbb{E}[\|\tilde{g}_t\|^2] \\ &\quad + \underbrace{\frac{L\eta\sigma^2 d}{B^2 T} \sum_{i=1}^T C_i^2}_{\text{variance}} + \underbrace{\frac{1}{T} \mathbb{E} \left[\sum_{t=1}^T \|g_t\| p' G \right]}_{\text{bias}} \end{aligned}$$

$$\leq \mathcal{O} \left(\frac{1}{T\eta} \right) + \mathcal{O}(\eta) + \mathcal{O} \left(\frac{T\eta d \ln(1/\delta)}{N^2 \epsilon^2} \right) + \underbrace{p' G^2}_{\text{bias}} \quad (9)$$

By setting $T\eta = \frac{N\epsilon}{\sqrt{d \ln(1/\delta)}}$ and $\eta = \frac{\sqrt{d \ln(1/\delta)}}{N\epsilon}$, we get:

$$\frac{1}{T} \mathbb{E} \left[\sum_{t=1}^T \|\nabla \mathcal{L}(\theta_t)\|^2 \right] \leq \mathcal{O} \left(\frac{\sqrt{d \ln(1/\delta)}}{N\epsilon} \right) + \text{bias}$$

The convergence analysis of DC-SGD-E is similar, with a minor modification as outlined in Equation (9):

$$\begin{aligned} &\frac{1}{T} \mathbb{E} \left[\sum_{t=1}^T \|\nabla \mathcal{L}(\theta_t)\|^2 \right] \\ &\leq \frac{1}{T\eta} (\mathbb{E}[\mathcal{L}(\theta_1)] - \mathbb{E}[\mathcal{L}(\theta_{T+1})]) + \frac{L\eta}{T} \sum_{t=1}^T \mathbb{E}[\|\tilde{g}_t\|^2] \\ &\quad + \frac{1}{T} \sum_{t=1}^T \left(\underbrace{L\eta \frac{\sigma^2 C_t^2 d}{B^2}}_{\text{Variance}} + \underbrace{\frac{1}{N} \sum_{i=1}^N G \max(\|g_{t,i}\| - C_t, 0)}_{\text{Bias}} \right) \\ &\leq \mathcal{O} \left(\frac{1}{T\eta} \right) + \mathcal{O}(\eta) + \mathcal{O} \left(\frac{T\eta d \ln(1/\delta)}{N^2 \epsilon^2} \right) + \underbrace{G^2}_{\text{bias}} \end{aligned} \quad (10)$$

We defer the detailed proof of the above analysis in the **Supplemental Material**. In the above convergence analysis, since DC-SGD-P and DC-SGD-E only differ at the setting of C_t during training, they have similar analyses and finally get the same result due to our analysis's loose upper bound of convergence. The result of in Theorem 3 is consistent with existing work [23], [24]. The non-vanishing is a biased term due to clipping operation, and previous work has shown DP-SGD with clipping suffers a constant regret in the worst case [46]. Except for the bias term, the result aligns with our expectations: (1) smaller ϵ and δ mean better privacy protection and stronger noise, making it harder for the model to converge; (2) larger d means a larger model, which is harder to converge.

Though the final convergence result is the same, DC-SGD-P and DC-SGD-E have different influences on the model convergence from different angles. DC-SGD-P ensures the probability that a gradient being clipped keeps constant during training, which can make the model update more stable. DC-SGD-E optimizes the expected squared error of a single gradient, which is similar to the variance and bias term in convergence analysis in the formulation. We believe that DC-SGD-E can also better balance the variance and bias in convergence analysis. Therefore, DC-SGD-E is more direct and reasonable.

V. EXPERIMENTAL SETUP

A. Investigated Baselines

We compare our proposed methods against three baseline approaches: vanilla DP-SGD, the method proposed by Andrew et al. [19], and the method proposed by Du et al. [23]. We apply their dynamic strategy for C under the RDP framework to make a fair comparison. And our methods has no limit on the privacy accounting, one can also use other

advanced privacy accounting techniques like [18] and [47]. The approach by Andrew et al. [19] adopts setting the clipping threshold to a specific percentile of gradient norms. While their algorithm is designed for federated learning, we evaluate it in the context of centralized learning. This method shares similarities with our DC-SGD-P approach as they both involve the introduction of the Percentile p . All of these methods work on different mainstream optimizers, so in this section, we abuse the notion. To illustrate our methods' advantage, we consider the time and privacy cost of hyperparameter tuning. In this work, we use grid search to tune the hyperparameter; we adopt LT [25] or RDP to account for the privacy cost of hyperparameter tuning.

B. Models and Datasets

1) *Datasets*: We use four benchmark datasets to measure the privacy-utility trade-off: CIFAR10 [31], MNIST [30], SVHN [32], and QNLI [33]. The first three are for computer vision tasks, and the last one is for natural language inference tasks. We defer the detailed in **Supplemental Material**.

2) *Models*: In our experiments, we use a small CNN with two convolution layers (28, 058 parameters), ResNet18 [34] (about 12M parameters), and ResNet34 [34] (about 22M parameters) for CV tasks. They are trained from scratch. Since the batch normalization in ResNet is unsupported under DP, we replaced it with group normalization. We use a pre-trained BERT-base [35] for the NLP task, which is provided in the huggingface transformers repository [48]. Since the model is pre-trained, we just fine-tune the last encoder, the Bert pooler, and the classifier. Therefore, we just need to train about 8M parameters. We construct all the above models with PyTorch [49] and follow the default setting of PyTorch.

C. Implementations

Our experiments vary the privacy budget $\epsilon \in \{1, 2, 4, 8\}$ for different scenarios. The δ is set to $\frac{1}{|D|}$, where $|D|$ is the size of training set in all situation. We use RDP to account for the total privacy cost. By default, all the experiments use training batch size $B = 256$, and we set epochs to 20 for ResNet18 and ResNet34, 10 for small CNN, and 5 for BERT-base. We treat the train set as the private dataset and the test set as the public set, and all the performance results we report are the accuracy of the test set, omitting the % symbol.

We use two types of optimizers in this section. The first is SGD with momentum, and we set the momentum to 0.9. This one needs to tune the learning rate η carefully. The other one is Adam [29], we use the default setting of optimizer parameters ($\eta = 0.001, \beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$) since the default setting works well in previous work [50].

The rest is the unique parameter of different methods. For DP-SGD, we need to tune C . For DC-SGD-P and DC-SGD-E, as the analysis in Section III, by default, we set $C_0 = 1, \sigma_H = 5, b = 20$ and do not need to tune them, where $\sigma_H = 5$ is due to in most experiments (σ is less than 1.2) it has least influence on the gradient noise according to Theorem 2. We set $R_0 = 1$ for DC-SGD-P and $R_0 = b$ for DC-SGD-E. **All these parameters do not need to be tuned, and they work among**

TABLE I
TIME COST OF HYPERPARAMETER TUNING OF DIFFERENT METHODS ON DIFFERENT MODEL ARCHITECTURES

Model	DP-SGD	DC-SGD-P	DC-SGD-E	Andrew [19]	Du [23]
CNN	3m6s×10	3m35s×9	3m34s	3m10s×45	3m8s×90
ResNet18	12m15s×10	12m30s×9	12m46s	12m20s×45	12m18s×90
ResNet34	23m54s×10	25m10s×9	26m12s	24m53s×45	23m50s×90
BERT	17m30s×10	20m44s×9	20m57s	19m58s×45	17m32s×90

all scenarios, like the default setting of Adam optimizer. Therefore, DC-SGD-E does not need to tune any parameter and DC-SGD-P only need to tune p . For the method of Andrew et al. [19]. We follow their setting: $\sigma_b = B/20, C_0 = 1$, and tune percentile p and learning rate of C : η_C . For the method of Du et al. [23]. We only apply their strategy to the adaptive clipping threshold. Therefore, we need to tune C_0, ρ_C . We defer the hyperparameter grid and the candidates of different hyperparameters in the **Supplemental Material**.

VI. EXPERIMENT RESULTS AND ANALYSIS

A. Running Time

We conduct experiments to illustrate the time cost of the training process, including hyperparameter tuning. We employ vanilla grid search for hyperparameter tuning, making the time cost equivalent to the product of the count of hyperparameter combinations and the time cost of a single training. We evaluate the time cost using the four mentioned model architectures on different training datasets: MNIST, CIFAR10, SVHN, and QNLI. The time cost of a single training is tested five times, and the average value, multiplied by the count of combinations, is considered the final result. In practical scenarios, adaptive optimizers such as Adam are commonly used to reduce time costs, as supported by previous work [50]. We also use the Adam optimizer and do not tune the learning rate (η).

Table I presents the experimental results. Notably, both DC-SGD-P and DC-SGD-E involve additional computations to adjust the hyperparameter C , resulting in slightly higher time costs per single training iteration compared to DP-SGD. A similar pattern is observed with the method proposed by Andrew et al., although its increase in time cost is comparatively smaller due to its simpler computational processes. In contrast, the approach introduced by Du et al. exhibits a time cost very close to DP-SGD, as they solely update the hyperparameter C based on iteration numbers.

These methods exhibit comparable single-training time costs. The decisive factor lies in the count of potential hyperparameter combinations associated with hyperparameters that require adjustment. Both Andrew's method and Du's method introduce additional hyperparameters that necessitate tuning, leading to an exceptionally high time cost. In contrast, DC-SGD-E requires no hyperparameter tuning and achieves approximately a $9\times$ speedup compared to DP-SGD. While DC-SGD-P requires tuning for the parameter p , and the quantities of p and C are similar, the time cost is essentially close to DP-SGD. However, p is generally easier to tune than C in practice, as discussed in Section VI-C. A similar scenario

applies to the SGD optimizer, which requires tuning for the learning rate η ; all five methods cannot circumvent this. So, the speedup results will be similar, and the actual time cost will increase due to the larger hyperparameter grid.

We emphasize that the analysis above specifically applies to our experimental setup, given that the number of candidate hyperparameters, a user decision greatly influences the time cost. Nevertheless, our method consistently reduces the number of hyperparameters requiring tuning, providing a clear advantage in terms of hyperparameter tuning time costs.

B. Privacy-Performance Trade-off

In this subsection, we analyze the utility of our methods, assessing the performance of the final outputted DP model from two perspectives: one considering the privacy cost of hyperparameter tuning and the other not.

1) *Considering the Privacy Cost of Tuning*: Previous studies have demonstrated that hyperparameter tuning introduces additional privacy costs [18], [25], [26]. In this subsection, we employ two methods to quantify the overall privacy cost. The first method involves using the LT algorithm [25], and the second method employs grid search, treating hyperparameter tuning as a sequential composition and applying Renyi Differential Privacy (RDP). According to previous research [50], these two methods are suitable for different scenarios, with LT performing better in cases involving a large number of training executions, while RDP performs better in the opposite scenario. Our experiment compares the performance of the best model obtained under the same predefined total privacy budget. For MNIST, where $\epsilon = 1$ is deemed too small, we use $\epsilon = 8$ instead. Additionally, due to the potentially large values of σ in DC-SGD-P, we adjust the σ_H accordingly (setting it to 8 for $2 \leq \sigma \leq 3$, and 12 for $\sigma > 3$) to ensure minimal impact on σ , thus incurring no hyperparameter tuning cost.

For the privacy analysis of LT in Theorem 1, following [50], we set an extremely small δ_2 (10^{-20}) and $\gamma = 1/2G$, where G represents the count of possible hyperparameter combinations. This choice implies that the expected running time of LT is $2G$, ensuring that we can practically explore almost all hyperparameter combinations. Under these conditions, we can derive the privacy cost for a single training. In the case of RDP, where the grid search involves repeatedly training G models, the privacy cost for a single training can be obtained through G -composition.

When considering the privacy cost of hyperparameter tuning, one will choose an optimizer that needs less hyperparameter to ensure the model performance. Therefore, we conduct experiments on the Adam optimizer as Mohapatra et al. [50] suggested Table II and Table III show the result using LT and RDP separately. DC-SGD-E does not need to tune any hyperparameter, and it will only train the model once with the total privacy cost. Therefore, it has the same performance in both situations.

In the results obtained using LT, DC-SGD-E demonstrates strong performance on CIFAR10 and SVHN, attributed to the complexity of these datasets and their sensitivity to noise levels. However, its improvement on the MNIST dataset is not as significant, as MNIST and CNN are relatively simple, and the

TABLE II
THE ACCURACY COMPARISON OF DIFFERENT METHODS ON ADAM OPTIMIZER USING LT [25] TO ACCOUNT FOR THE PRIVACY COST OF HYPERPARAMETER TUNING

Models	Dataset	ϵ	DP-SGD	DC-SGD-P	DC-SGD-E	Andrew [19]	Du [23]
CNN	MNIST	2	92.39	92.06	94.18	92.12	91.46
		4	93.84	93.75	94.70	93.54	93.65
		8	95.29	94.54	94.95	94.99	94.86
BERT	QNLI	2	68.68	67.60	74.08	67.12	67.49
		4	70.29	69.60	75.56	70.02	69.72
		8	72.27	71.21	76.42	71.96	71.68
ResNet18	CIFAR10	2	30.43	30.09	48.56	29.31	29.43
		4	39.25	39.83	51.91	38.87	40.08
		8	44.85	45.50	54.06	45.51	45.19
	SVHN	2	61.87	60.81	80.89	57.21	53.72
		4	74.26	75.35	82.64	74.96	75.56
		8	78.18	79.58	83.29	79.31	78.45
ResNet34	CIFAR10	2	24.38	24.12	43.92	23.16	22.93
		4	33.97	34.18	47.10	34.54	34.78
		8	39.75	39.68	50.01	40.71	40.99
	SVHN	2	49.96	49.01	79.19	48.67	43.78
		4	70.87	71.29	81.54	70.96	70.21
		8	74.88	76.22	82.61	75.68	75.94

TABLE III
THE ACCURACY COMPARISON OF DIFFERENT METHODS ON ADAM OPTIMIZER USING RDP TO ACCOUNT FOR THE PRIVACY COST OF HYPERPARAMETER TUNING

Models	Dataset	ϵ	DP-SGD	DC-SGD-P	DC-SGD-E	Andrew [19]	Du [23]
CNN	MNIST	2	94.24	94.18	94.18	92.37	91.11
		4	95.08	94.91	94.70	93.82	93.42
		8	95.99	95.78	94.95	94.67	94.06
BERT	QNLI	2	72.76	72.67	74.08	68.83	68.61
		4	74.28	73.13	75.56	71.74	70.36
		8	75.03	75.16	76.42	73.55	73.16
ResNet18	CIFAR10	2	38.76	39.11	48.56	31.47	28.44
		4	44.36	45.10	51.91	38.79	35.94
		8	49.25	49.41	54.06	45.30	41.79
	SVHN	2	73.88	75.51	80.89	65.42	54.91
		4	78.86	79.62	82.64	74.75	69.65
		8	80.83	81.33	83.29	79.03	76.05
ResNet34	CIFAR10	2	33.30	33.75	43.92	25.73	24.35
		4	40.51	40.65	47.10	33.26	30.99
		8	43.57	44.38	50.01	39.71	37.28
	SVHN	2	69.76	71.89	79.19	50.71	40.82
		4	75.95	77.09	81.54	70.66	62.28
		8	78.01	79.75	82.61	75.66	72.59

noise level has a lesser impact on the final model performance. Moreover, a smaller ϵ leads to a more pronounced improvement for DC-SGD-E compared to other methods, as a small total privacy budget causes a more significant difference in the noise multiplier among different methods. On the other hand, for the remaining methods, LT results in an inflation of ϵ by at least $3\times$. The other four methods exhibit similar performance, as LT is not highly sensitive to the count of hyperparameter combinations. Nevertheless, it's crucial to note that DP-SGD and DC-SGD-P have significantly lower time costs compared to the methods of Andrew et al. and Du et al. In summary, with LT, DC-SGD-E performs well across various scenarios, especially on complex datasets with a small privacy budget, and DC-SGD-P achieves comparable or superior performance compared to other methods.

In the results obtained using RDP, DC-SGD-E continues to exhibit the best performance, and the analysis for DC-SGD-E is consistent with previous observations. However, the differences in performance among the remaining methods become more pronounced as the total privacy cost is related to the count of hyperparameter combinations. Overall, the ranking in terms of performance is DC-SGD-P < DP-SGD

TABLE IV
THE ACCURACY COMPARISON OF DIFFERENT METHODS FOR ADAM OPTIMIZER WITHOUT THE PRIVACY COST OF TUNING

Models	Dataset	ϵ	DP-SGD	DC-SGD-P	DC-SGD-E	Andrew [19]	Du [23]
CNN	MNIST	1	94.78 $\pm$ 0.21	94.72 $\pm$ 0.19	94.04 $\pm$ 0.30	94.69 $\pm$ 0.20	94.72 $\pm$ 0.22
		2	95.62 $\pm$ 0.15	95.08 $\pm$ 0.27	94.19 $\pm$ 0.87	95.45 $\pm$ 0.29	95.63 $\pm$ 0.24
		4	95.95 $\pm$ 0.11	95.60 $\pm$ 0.20	94.65 $\pm$ 0.16	96.16 $\pm$ 0.15	95.94 $\pm$ 0.15
BERT-base	QNLI	2	73.21 $\pm$ 0.48	73.21 $\pm$ 0.36	74.31 $\pm$ 0.38	73.40 $\pm$ 0.49	74.24 $\pm$ 0.36
		4	75.36 $\pm$ 0.22	74.35 $\pm$ 0.25	75.61 $\pm$ 0.21	74.29 $\pm$ 0.58	75.87 $\pm$ 0.29
		8	76.20 $\pm$ 0.53	75.65 $\pm$ 0.15	76.29 $\pm$ 0.39	75.49 $\pm$ 0.15	76.29 $\pm$ 0.54
ResNet18	CIFAR10	2	48.34 $\pm$ 0.45	48.63 $\pm$ 0.42	48.89 $\pm$ 0.48	49.21 $\pm$ 0.23	49.48 $\pm$ 0.81
		4	51.28 $\pm$ 0.16	51.46 $\pm$ 0.15	51.42 $\pm$ 0.38	51.49 $\pm$ 0.26	52.28 $\pm$ 0.64
		8	53.81 $\pm$ 0.46	54.18 $\pm$ 0.32	53.76 $\pm$ 0.27	54.30 $\pm$ 0.78	54.42 $\pm$ 0.32
	SVHN	2	80.05 $\pm$ 0.21	80.42 $\pm$ 0.28	81.39 $\pm$ 0.46	80.96 $\pm$ 0.08	81.07 $\pm$ 0.29
		4	81.03 $\pm$ 0.46	81.43 $\pm$ 0.22	82.45 $\pm$ 0.36	81.50 $\pm$ 0.29	82.15 $\pm$ 0.22
		8	81.92 $\pm$ 0.46	83.06 $\pm$ 0.13	83.44 $\pm$ 0.40	83.20 $\pm$ 0.23	83.07 $\pm$ 0.26
ResNet34	CIFAR10	2	43.72 $\pm$ 0.40	43.66 $\pm$ 0.38	44.07 $\pm$ 0.59	44.15 $\pm$ 0.28	44.52 $\pm$ 0.60
		4	46.89 $\pm$ 0.59	47.34 $\pm$ 0.26	47.30 $\pm$ 0.44	48.25 $\pm$ 0.42	47.75 $\pm$ 0.46
		8	49.79 $\pm$ 0.36	49.28 $\pm$ 0.44	50.10 $\pm$ 0.28	50.13 $\pm$ 0.24	50.26 $\pm$ 0.53
	SVHN	2	78.47 $\pm$ 0.33	78.90 $\pm$ 0.49	79.26 $\pm$ 0.28	79.24 $\pm$ 0.31	79.06 $\pm$ 0.40
		4	79.70 $\pm$ 0.47	80.08 $\pm$ 0.18	81.67 $\pm$ 0.34	80.53 $\pm$ 0.33	80.37 $\pm$ 0.28
		8	80.66 $\pm$ 0.30	81.81 $\pm$ 0.39	82.47 $\pm$ 0.46	82.12 $\pm$ 0.18	81.60 $\pm$ 0.30

TABLE V
THE ACCURACY COMPARISON OF DIFFERENT METHODS FOR SGD OPTIMIZER WITHOUT THE PRIVACY COST OF TUNING

Models	Dataset	ϵ	DP-SGD	DC-SGD-P	DC-SGD-E	Andrew [19]	Du [23]
CNN	MNIST	1	94.51 $\pm$ 0.19	93.98 $\pm$ 0.25	94.06 $\pm$ 0.35	94.16 $\pm$ 0.17	95.23 $\pm$ 0.30
		2	95.86 $\pm$ 0.08	94.86 $\pm$ 0.15	94.34 $\pm$ 0.14	95.53 $\pm$ 0.24	96.10 $\pm$ 0.41
		4	96.25 $\pm$ 0.20	95.89 $\pm$ 0.18	94.64 $\pm$ 0.14	96.34 $\pm$ 0.07	96.56 $\pm$ 0.21
BERT-base	QNLI	2	73.48 $\pm$ 0.22	73.13 $\pm$ 0.40	74.17 $\pm$ 0.84	73.68 $\pm$ 0.80	74.20 $\pm$ 0.48
		4	75.29 $\pm$ 0.69	74.94 $\pm$ 0.59	75.33 $\pm$ 0.89	74.39 $\pm$ 0.88	75.58 $\pm$ 0.74
		8	76.70 $\pm$ 0.73	75.81 $\pm$ 0.54	76.74 $\pm$ 0.71	76.12 $\pm$ 0.48	76.65 $\pm$ 0.82
ResNet18	CIFAR10	2	48.88 $\pm$ 0.60	49.01 $\pm$ 0.27	48.05 $\pm$ 0.11	48.98 $\pm$ 0.68	50.08 $\pm$ 0.23
		4	51.19 $\pm$ 0.50	52.16 $\pm$ 0.55	51.62 $\pm$ 0.43	52.35 $\pm$ 0.10	52.98 $\pm$ 0.30
		8	54.13 $\pm$ 0.27	53.81 $\pm$ 0.33	53.91 $\pm$ 0.25	53.83 $\pm$ 0.21	54.74 $\pm$ 0.29
	SVHN	2	80.47 $\pm$ 0.41	80.74 $\pm$ 0.15	81.76 $\pm$ 0.24	80.69 $\pm$ 0.32	81.48 $\pm$ 0.29
		4	81.80 $\pm$ 0.13	81.80 $\pm$ 0.16	82.77 $\pm$ 0.30	81.96 $\pm$ 0.15	82.52 $\pm$ 0.06
		8	82.48 $\pm$ 0.27	82.23 $\pm$ 0.10	83.45 $\pm$ 0.22	82.76 $\pm$ 0.06	82.90 $\pm$ 0.17
ResNet34	CIFAR10	2	44.24 $\pm$ 0.35	45.41 $\pm$ 0.76	44.00 $\pm$ 0.49	45.49 $\pm$ 0.31	45.65 $\pm$ 0.40
		4	47.43 $\pm$ 0.33	47.80 $\pm$ 0.29	47.48 $\pm$ 0.33	47.75 $\pm$ 0.45	49.49 $\pm$ 0.42
		8	49.60 $\pm$ 0.36	50.70 $\pm$ 0.65	49.73 $\pm$ 0.51	51.08 $\pm$ 0.79	51.36 $\pm$ 0.38
	SVHN	2	78.38 $\pm$ 0.11	78.60 $\pm$ 0.53	79.70 $\pm$ 0.75	79.49 $\pm$ 0.58	79.88 $\pm$ 0.25
		4	79.99 $\pm$ 0.48	80.72 $\pm$ 0.35	81.73 $\pm$ 0.20	80.91 $\pm$ 0.25	81.40 $\pm$ 0.27
		8	80.70 $\pm$ 0.47	81.12 $\pm$ 0.41	82.39 $\pm$ 0.22	81.59 $\pm$ 0.21	82.05 $\pm$ 0.41

<Andrew<Du, aligning with the counts of hyperparameter combinations for each method.

In summary, regardless of the method used to measure the cost of hyperparameter tuning, both DC-SGD-P and DC-SGD-E benefit from a smaller scale of hyperparameter tuning, leading to certain performance improvements.

2) *Without Considering the Privacy Cost of Tuning:* Most existing work ignores the privacy cost of hyperparameter tuning [19], [38], [45], [51]. Therefore, we also conduct experiments under this situation to better compare our method with other existing approaches. Since we do not consider the privacy cost, we apply both Adam and SGD optimizers in this subsection. Additionally, we can apply a small privacy budget for MNIST.

Table IV and Table V illustrate the experiment results separately. We test different hyperparameter combinations, identify the best one, repeat the training 5 times, and report the average accuracy. From the results, we draw the following analysis: (1) In general, both DC-SGD-P and DC-SGD-E perform well with both optimizers, achieving performance close to or better

than other existing algorithms. Moreover, our methods require fewer hyperparameter adjustments and are more time-efficient. (2) DC-SGD-P shows performance close to the method of Andrew et al. This is because DC-SGD-P and DC-SGD-E both adopt the concept of Percentile, but DC-SGD-P tunes fewer hyperparameters. (3) The method of Du et al. can achieve the best performance under most settings, but it comes with the cost of tuning the most hyperparameters. (4) DC-SGD-E does not perform as well on MNIST; we attribute this to MNIST and CNN being simple enough to tolerate stronger noise, rendering the variance term in the expectation-squared error less sensitive to the impact of noise.

Under the situation of not considering the privacy cost of tuning, one can still choose DC-SGD-E and DC-SGD-P, as they achieve comparable or better performance compared to other methods with less tuning time cost.

C. The Influence of Parameters

To analyze the influence of p, b, σ_H in DC-SGD-P and DC-SGD-E separately and illustrate that we do not need to

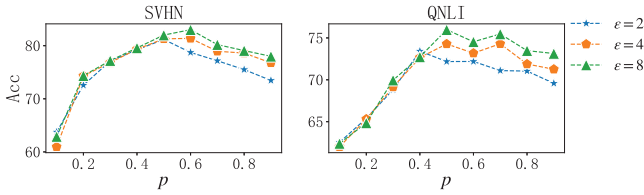


Fig. 5. The accuracy of different p for DC-SGD-P on SVHN and QNLI.

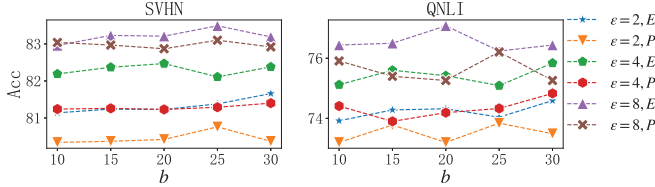


Fig. 6. The accuracy of b for DC-SGD-P (P) and DC-SGD-E (E) on SVHN and QNLI.

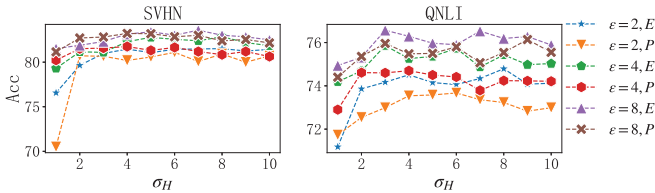


Fig. 7. The accuracy of different σ_H for DC-SGD-P (P) and DC-SGD-E (E) on SVHN and QNLI.

tune b, σ_H , we conduct experiments on SVHN (ResNet18) and QNLI (BERT-base) with the Adam optimizer. More experiments on other datasets can be found in **Supplemental Material**. We use the best p of different scenarios from Section VI-B.2 as the default value for DC-SGD-P in this subsection. When evaluating the impact of one parameter, the other parameters are fixed at their default values.

Figure 5 illustrates the influence of percentile p on accuracy. p significantly impacts the final model performance and needs to be chosen carefully. The best p differs when ϵ and the dataset change, but the best p is close among different privacy budgets. It seems that the more complex the task is, the smaller p we should choose. Besides, the best hyperparameter combination from Section VI-B.2 shows that the best p remains close for a dataset on different models (e.g., SVHN on ResNet18 and ResNet34). All the above indicates that the choice of p follows a pattern.

Figure 6 shows the accuracy of different b . The choice of b has less impact on the model accuracy, considering the randomness during training. Therefore, we do not need to tune b . Figure 7 shows the variation in accuracy under different σ_H . We can find that the model performance is not sensitive to σ_H except for the situation where σ_H is small and it incurs a significant increase in the noise multiplier of the gradient, e.g., $\sigma_H = 1, \epsilon = 2$ in this subsection. Therefore, one only needs to choose σ_H by ensuring that it won't cause a large increase in σ_T and it does not incur extra tuning costs.

VII. RELATED WORK

A. DP-SGD and Variants

Abadi et al. [18] introduced DP-SGD, a foundational method for training deep learning models with DP. They suggested using the median of unclipped gradient norms as the clipping threshold but lacked a method with formal privacy guarantees. For privacy analysis, they used Moments Accountant (MA), later improved by Mironov [42] with Rényi Differential Privacy (RDP). Efforts to enhance DP-SGD include Papernot et al. [52], who replaced ReLU with tempered sigmoids for better performance, and Yu et al. [44], [53], who reduced gradient dimensionality to lower noise in large models. Yu et al. [37] also introduced a dynamic privacy budget allocator under Concentrated Differential Privacy (CDP). Xiao et al. [43] proposed ModelMix, using random aggregation of intermediate model states for updates. Liu et al. [54] decomposed gradients into orthogonal and parallel components, focusing the privacy budget on the orthogonal part. Sha et al. [55] optimized DP-SGD under a heavy-tailed distribution, applying different clipping thresholds for the heavy tail and light body of gradients. Despite these improvements, all methods still rely on clipping to bound gradient sensitivity.

B. Adaptive Clipping

Recent studies have tackled the challenges associated with the clipping threshold by focusing on its adaptation. Pichapati et al. [27] introduced a coordinate-wise adaptive clipping method to reduce gradient noise. Andrew et al. [19] applied percentile-based updates in federated learning with DP-FedAvg, using a geometric update scheme and a hyperparameter to control update speed. Golatkar et al. [22] used a public dataset for adaptive adjustment of C through percentiles, avoiding additional privacy costs, though finding a suitable public dataset remains challenging. Du et al. [23] dynamically adjusted both the clipping threshold and noise power based on iteration number under Gaussian Differential Privacy. However, their method requires an initial C tailored to specific settings due to the lack of norm distribution data and involves hyperparameters that require tuning. In contrast, our DC-SGD method directly accesses the norm distribution under DP to determine the optimal C , eliminating the need for a public dataset and reducing dependency on hyperparameters.

C. Normalization-Based Clipping

The second approach eliminates the need for a specific clipping threshold. De et al. [20] proposed normalizing the gradient using C , effectively incorporating it into the learning rate η . They demonstrated that using a small C (e.g., $C = 1$) consistently yields strong results, allowing the focus to shift to tuning η . Similarly, Bu et al. [21] and Yang et al. [56] introduced normalization-based clipping functions that combine η and C , thereby eliminating the need for a separate clipping threshold. These functions normalize the influence of each sample to 1, ensuring bounded sensitivity. While these methods alter the traditional clipping strategy within

DP-SGD, our work retains the standard approach, focusing on dynamically adjusting C . Although these approaches are orthogonal to ours, we conducted comparative experiments with AutoClip, proposed by Bu et al. [21], with results detailed in the Supplemental Material.

Amin et al. [57], Chen et al. [24], and Zhang et al. [58] explored the impact of clipping on convergence. Liu et al. [25], Mohapatra et al. [50], and Papernot et al. [26] underscored the privacy costs of hyperparameter tuning and proposed mitigation strategies. Our work focuses on minimizing the time and privacy costs of hyperparameter tuning. DC-SGD reduces privacy costs during training while requiring fewer hyperparameter combinations while maintaining strong model performance. Recently, Liu et al. [59] and Zhang et al. [60] combined DP-SGD and Zeroth-Order Optimization to tackle the challenges of fine-tuning LLM with DP: high time and memory cost and bad model utility. And their works still need a clipping threshold to clip the gradient. We think we can combine their works with our methods since hyperparameter tuning is very expensive in the LLM scenario. We leave this as our future research direction.

VIII. CONCLUSION

This paper introduces DC-SGD, a novel differentially private training solution with an adaptive dynamic clipping threshold. Leveraging the gradient norm distribution, we propose two mechanisms for dynamically setting the clipping threshold: DC-SGD-P uses a percentile p , while DC-SGD-E optimizes the expected squared error. We formally establish privacy and convergence guarantees for DC-SGD and validate its effectiveness through extensive experiments on benchmark datasets across various deep models. The results demonstrate that DC-SGD outperforms or matches DP-SGD while reducing hyperparameter tuning costs. Additionally, DC-SGD integrates seamlessly with both SGD and Adam optimizers.

ACKNOWLEDGMENT

The authors sincerely thank all the anonymous reviewers for their valuable feedback and constructive suggestions.

REFERENCES

- [1] Y. Zhao, Q. Ye, W. Wu, C. Shen, and F. Wan, "Generative prompt model for weakly supervised object localization," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2023, pp. 6351–6361.
- [2] P. Wang, E. Fan, and P. Wang, "Comparative analysis of image classification algorithms based on traditional machine learning and deep learning," *Pattern Recognit. Lett.*, vol. 141, pp. 61–67, Jan. 2021.
- [3] K. Zhou, Y. Yang, T. Hospedales, and T. Xiang, "Deep domain-adversarial image generation for domain generalisation," in *Proc. AAAI Conf. Artif. Intell.*, vol. 34, no. 7, 2020, pp. 13025–13032.
- [4] J. Ho, C. Saharia, W. Chan, D. J. Fleet, M. Norouzi, and T. Salimans, "Cascaded diffusion models for high fidelity image generation," *J. Mach. Learn. Res.*, vol. 23, pp. 2249–2281, Jan. 2022.
- [5] C. Gong et al., "Baffle: Hiding backdoors in offline reinforcement learning datasets," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2024, pp. 2086–2104.
- [6] D. W. Otter, J. R. Medina, and J. K. Kalita, "A survey of the usages of deep learning for natural language processing," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 2, pp. 604–624, Apr. 2020.
- [7] M. Popel et al., "Transforming machine translation: A deep learning system reaches news translation quality comparable to human professionals," *Nature Commun.*, vol. 11, no. 1, p. 4381, Sep. 2020.
- [8] T. Iqbal and S. Qureshi, "The survey: Text generation models in deep learning," *J. King Saud Univ.-Comput. Inf. Sci.*, vol. 34, no. 6, pp. 2515–2528, Jun. 2022.
- [9] J. Li, A. Sun, J. Han, and C. Li, "A survey on deep learning for named entity recognition," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 1, pp. 50–70, Mar. 2020.
- [10] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2017, pp. 3–18.
- [11] C. A. Choquette-Choo, F. Tramer, N. Carlini, and N. Papernot, "Label-only membership inference attacks," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 1964–1974.
- [12] J. Hayes, L. Melis, G. Danezis, and E. De Cristofaro, "LOGAN: Membership inference attacks against generative models," 2017, *arXiv:1705.07663*.
- [13] M. Fredrikson, S. Jha, and T. Ristenpart, "Model inversion attacks that exploit confidence information and basic countermeasures," in *Proc. 22nd ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2015, pp. 1322–1333.
- [14] C. Gong et al., "Curiosity-driven and victim-aware adversarial policies," in *Proc. 38th Annu. Comput. Secur. Appl. Conf.*, Dec. 2022, pp. 186–200.
- [15] K. Li, C. Gong, Z. Li, Y. Zhao, X. Hou, and T. Wang, "PrivImage: Differentially private synthetic image generation using diffusion models with semantic-aware pretraining," 2023, *arXiv:2311.12850*.
- [16] C. Gong, X. Zhang, and Y. Niu, "Identification of epilepsy from intracranial EEG signals by using different neural network models," *Comput. Biol. Chem.*, vol. 87, Aug. 2020, Art. no. 107310.
- [17] Y. Pang, T. Wang, X. Kang, M. Huai, and Y. Zhang, "White-box membership inference attacks against diffusion models," 2023, *arXiv:2308.06405*.
- [18] M. Abadi et al., "Deep learning with differential privacy," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security*, 2016, pp. 308–318.
- [19] G. Andrew, O. Thakkar, H. B. McMahan, and S. Ramaswamy, "Differentially private learning with adaptive clipping," in *Proc. Annu. Conf. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 17455–17466.
- [20] S. De, L. Berrada, J. Hayes, S. L. Smith, and B. Balle, "Unlocking high-accuracy differentially private image classification through scale," 2022, *arXiv:2204.13650*.
- [21] Z. Bu, Y.-X. Wang, S. Zha, and G. Karypis, "Automatic clipping: Differentially private deep learning made easier and stronger," 2022, *arXiv:2206.07136*.
- [22] A. Golatkar, A. Achille, Y.-X. Wang, A. Roth, M. Kearns, and S. Soatto, "Mixed differential privacy in computer vision," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 8366–8376.
- [23] J. Du, S. Li, X. Chen, S. Chen, and M. Hong, "Dynamic differential-privacy preserving SGD," 2021, *arXiv:2111.00173*.
- [24] X. Chen, S. Z. Wu, and M. Hong, "Understanding gradient clipping in private SGD: A geometric perspective," in *Proc. Annu. Conf. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 13773–13782.
- [25] J. Liu and K. Talwar, "Private selection from private candidates," in *Proc. 51st Annu. ACM SIGACT Symp. Theory Comput.*, Jun. 2019, pp. 298–309.
- [26] N. Papernot and T. Steinke, "Hyperparameter tuning with Renyi differential privacy," 2021, *arXiv:2110.03620*.
- [27] V. Pichapati, A. Theertha Suresh, F. X. Yu, S. J. Reddi, and S. Kumar, "AdaClip: Adaptive clipping for private SGD," 2019, *arXiv:1908.07643*.
- [28] J. Dong, A. Roth, and W. J. Su, "Gaussian differential privacy," 2019, *arXiv:1905.02383*.
- [29] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*.
- [30] Y. LeCun and C. Cortes. (2010). *MNIST Handwritten Digit Database*. [Online]. Available: <http://yann.lecun.com/exdb/mnist/>
- [31] A. Krizhevsky and G. Hinton, "Learning multiple layers of features from tiny images," Univ. Toronto, Toronto, ON, Canada, Tech. Rep., 2009.
- [32] Y. Netzer, T. Wang, A. Coates, A. Bissacco, B. Wu, and A. Y. Ng, "Reading digits in natural images with unsupervised feature learning," in *Proc. NIPS Workshop Deep Learn. unsupervised Feature Learn.*, vol. 2, 2011, pp. 1–9.
- [33] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. R. Bowman, "GLUE: A multi-task benchmark and analysis platform for natural language understanding," 2018, *arXiv:1804.07461*.
- [34] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.

- [35] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," 2018, *arXiv:1810.04805*.
- [36] I. Mironov, "Rényi differential privacy," in *Proc. IEEE 30th Comput. Secur. Found. Symp. (CSF)*, Aug. 2017, pp. 263–275.
- [37] L. Yu, L. Liu, C. Pu, M. E. Gursoy, and S. Truex, "Differentially private model publishing for deep learning," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2019, pp. 332–349.
- [38] F. Tramèr and D. Boneh, "Differentially private learning needs better features (or much more data)," 2020, *arXiv:2011.11660*.
- [39] C. Dwork, F. McSherry, K. Nissim, and A. Smith, "Calibrating noise to sensitivity in private data analysis," in *Proc. Theory Cryptogr. Conf.*, 2006, pp. 265–284.
- [40] H. Brendan McMahan, D. Ramage, K. Talwar, and L. Zhang, "Learning differentially private recurrent language models," 2017, *arXiv:1710.06963*.
- [41] B. Balle, G. Barthe, M. Gaboardi, J. Hsu, and T. Sato, "Hypothesis testing interpretations and Rényi differential privacy," in *Proc. Int. Conf. Artif. Intell. Statist. (AISTATS)*, Aug. 2020, pp. 2496–2506.
- [42] I. Mironov, K. Talwar, and L. Zhang, "Rényi differential privacy of the sampled Gaussian mechanism," 2019, *arXiv:1908.10530*.
- [43] H. Xiao, J. Wan, and S. Devadas, "Differentially private deep learning with ModelMix," 2022, *arXiv:2210.03843*.
- [44] D. Yu, H. Zhang, W. Chen, and T.-Y. Liu, "Do not let privacy overbill utility: Gradient embedding perturbation for private learning," 2021, *arXiv:2102.12677*.
- [45] J. Wei, E. Bao, X. Xiao, and Y. Yang, "DPIS: An enhanced mechanism for differentially private SGD with importance sampling," 2022, *arXiv:2210.09634*.
- [46] S. Song, T. Steinke, O. Thakkar, and A. Thakurta, "Evading curse of dimensionality in unconstrained private GLMs via private gradient descent," 2020, *arXiv:2006.06783*.
- [47] S. Gopi, Y. T. Lee, and L. Wutschitz, "Numerical composition of differential privacy," in *Proc. Adv. Neural Inf. Process. Syst.*, Jan. 2021, pp. 11631–11642.
- [48] T. Wolf et al., "Transformers: State-of-the-art natural language processing," in *Proc. Conf. Empirical Methods Natural Lang. Syst. Demonstration*, Oct. 2020, pp. 38–45. [Online]. Available: <https://aclanthology.org/>
- [49] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, Jan. 2019, pp. 1–14.
- [50] S. Mohapatra, S. Sasy, X. He, G. Kamath, and O. Thakkar, "The role of adaptive optimizers for honest private hyperparameter selection," in *Proc. AAAI Conf. Artif. Intell.*, Jun. 2022, vol. 36, no. 7, pp. 7806–7813.
- [51] D. Yu et al., "Differentially private fine-tuning of language models," 2021, *arXiv:2110.06500*.
- [52] N. Papernot, A. Thakurta, S. Song, S. Chien, and Ú. Erlingsson, "Tempered sigmoid activations for deep learning with differential privacy," in *Proc. AAAI Conf. Artif. Intell.*, May 2021, vol. 35, no. 10, pp. 9312–9321.
- [53] D. Yu, H. Zhang, W. Chen, J. Yin, and T.-Y. Liu, "Large scale private learning via low-rank reparametrization," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 12208–12218.
- [54] Y. Liu, L. Xiong, Y. Liu, Y. Gu, R. Liu, and H. Chen, "DPDR: Gradient decomposition and reconstruction for differentially private deep learning," 2024, *arXiv:2406.02744*.
- [55] H. Sha, Y. Cao, Y. Liu, Y. Wu, R. Liu, and H. Chen, "Clip body and tail separately: High probability guarantees for DPSGD with heavy tails," 2024, *arXiv:2405.17529*.
- [56] X. Yang, H. Zhang, W. Chen, and T.-Y. Liu, "Normalized/Clipped SGD with perturbation for differentially private non-convex optimization," 2022, *arXiv:2206.13033*.
- [57] K. Amin, A. Kulesza, A. Munoz, and S. Vassilvitskii, "Bounding user contributions: A bias-variance trade-off in differential privacy," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2019, pp. 263–271.
- [58] X. Zhang, X. Chen, M. Hong, Z. S. Wu, and J. Yi, "Understanding clipping for federated learning: Convergence and client-level differential privacy," in *Proc. Int. Conf. Mach. Learn. (ICML)*, Jan. 2021, pp. 1–20.
- [59] Z. Liu et al., "Differentially private zeroth-order methods for scalable large language model finetuning," 2024, *arXiv:2402.07818*.
- [60] L. Zhang, B. Li, K. K. Thekumparampil, S. Oh, and N. He, "DPZero: Private fine-tuning of language models without backpropagation," in *Proc. 41st Int. Conf. Mach. Learn.*, 2024, pp. 1–41.



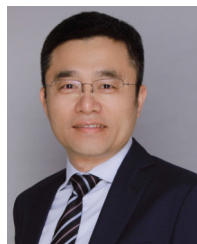
Chengkun Wei received the Ph.D. degree in computer science from Zhejiang University. He is a ZJU 100-Young Professor with the College of Computer Science and Technology, Zhejiang University. He has published over ten papers in prestigious international journals and conferences, including CCS, NDSS, SIGCOMM, ATC, HPCA, IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING, and IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY. His research interests encompass data security (differential privacy and AI security) and architecture design for cloud computing (network vSwitch and trusted execution environment operating system design). He received the NDSS 2024 Distinguished Paper Award. He is a reviewer of several international journals.



Weixian Li received the bachelor's and master's degrees in computer science and technology from Zhejiang University, in 2021 and 2024, respectively. He is an Algorithm Engineer with the Ant Group, working on applicable privacy protection of LLM, differential privacy, and marketing algorithm.



Chen Gong received the bachelor's degree from China University of Geosciences, Beijing, in 2020, and the master's degree from the Institute of Automation, Chinese Academy of Sciences, in 2023. He is currently pursuing the Ph.D. degree with the University of Virginia. His research interests include AI security, differential privacy, and reinforcement learning.



Wenzhi Chen (Member, IEEE) received the Ph.D. degree from the College of Computer Science and Engineering, Zhejiang University. He is a Professor with the College of Computer Science and Technology, Zhejiang University, and the Director of the Information Technology Center, Zhejiang University. He was the Vice Dean of the College of Computer Science and Technology. His current research interests include computer architecture, computer system software, LLM application and information security. He is a member of ACM and the ACM Education Council. His research has been honored with the Distinguished Paper Award at NDSS 2024.